```
    {
        std::shared_ptr<node> old_head=std::atomic_load(&head);
        while(old_head &&amp;
!std::atomic_compare_exchange_weak(&head,
                  &old_head,old_head->next));
        return old_head ? old_head->data : std::shared_ptr<T>();
    }
};
```

In the probable case that your `std::shared_ptr<>` implementation isn't lock-free, you need to manage the reference counting manually.

One possible technique involves the use of not one but two reference counts for each node: an internal count and an external count. The sum of these values is the total number of references to the node. The external count is kept alongside the pointer to the node and is increased every time the pointer is read. When the reader is finished with the node, it decreases the *internal* count. A simple operation that reads the pointer will thus leave the external count increased by one and the internal count decreased by one when it's finished.

When the external count/pointer pairing is no longer required (that is, the node is no longer accessible from a location accessible to multiple threads), the internal count is increased by the value of the external count minus one and the external counter is discarded. Once the internal count is equal to zero, there are no outstanding references to the node and it can be safely deleted. It's still important to use atomic operations for updates of shared data. Let's now look at an implementation of a lock-free stack that uses this technique to ensure that the nodes are reclaimed only when it's safe to do so.

The following listing shows the internal data structure and the implementation of `push()`, which is nice and straightforward.

Listing 7.10. Pushing a node on a lock-free stack using split reference counts